

FastAPI 시작하기

라우팅

- 개념

- 클라이언트로부터 오는 HTTP 요청을 알맞은 로직(함수나 메서드)으로 연결하는 과정을 의미
- 즉, 특정 URL을 어떤 함수가 처리할지를 정의하는 것
- FastAPI에서는 데코레이터를 사용하여 간단하게 처리할 수 있음

- 기본 라우팅: @app.get("/")

- HTTP GET 메서드를 사용한 경우
- 다음 예시는 루트 URL(<http://127.0.0.1:8000/>)에 GET 요청을 하면 “Hello, FastAPI!”라는 응답을 보내는 예

```
from fastapi import FastAPI

app = FastAPI()

@app.get("/")
def read_root():
    return {"message": "Hello, FastAPI!"}
```

라우팅

- 경로 매개변수

- URL의 특정 부분을 변수로써 사용하여 동적으로 변할 수 있는 값을 처리할 때 사용
- 경로의 일부로 인식

```
@app.get("/items/{item_id}")  
def read_item(item_id):  
    return {"item_id": item_id}
```

- /items/{item_id}에서 {item_id} 이 경로 매개변수
- 사용자가 방문하는 URL의 해당 부분에 있는 값을 int 타입으로 read_item() 함수에 전달
- 복수의 경로 매개변수 사용

```
@app.get("/users/{user_id}/items/{item_name}")  
def read_user_item(user_id, item_name):  
    # 여기서 user_id와 item_name은 문자열로 전달됨.  
    return {"user_id": user_id, "item_name": item_name}
```

- {user_id}와 {item_name}이 경로 매개변수

라우팅

- 쿼리 매개변수

- URL의 경로 이후 ?로 시작되는 부분에 정의되며, 키-값 쌍의 형태로 정보를 전달하는 데 사용
- 필터링, 정렬, 페이지네이션 등과 같이 요청을 더 세부적으로 조정할 필요가 있을 때 활용
- 쿼리 매개변수는 선택적이며, 동일한 경로에 대해서 다양한 연산을 가능하게 함

```
@app.get("/items/")
def read_items(skip, limit):
    return {"skip": skip, "limit": limit}
```

- 요청 : `/items/?skip=5&limit=5`와 같이 요청을 서버로 보내면, `skip`과 `limit` 이 쿼리 매개변수

```
@app.get("/items/")
def read_items(skip = 0, limit = 10):
    return {"skip": skip, "limit": limit}
```

- 쿼리 매개변수에 기본값을 설정하면 클라이언트가 이 값들을 제공하지 않아도 기본값인 0과 10이 적용.

curl을 사용한 테스트

- curl은 Client URL의 약자
- 다양한 프로토콜을 지원하는 명령행 기반의 네트워크 도구
- 주로 웹 서버와의 상호작용을 위해 사용되며, HTTP, HTTPS, FTP 등 다양한 프로토콜을 지원함
- 테스트 예시

- `curl http://127.0.0.1:8000/`

- `curl http://127.0.0.1:8000/items/5`

- `curl "http://127.0.0.1:8000/items/?skip=20&limit=2"`

- `curl "http://127.0.0.1:8000/items/"`

타입 힌트(Type Hint)

- 프로그래밍에서 변수나 함수의 예상 타입을 명시적으로 표시하는 기술
- FastAPI는 파이썬의 타입 힌트를 사용하여 요청을 검증하고, 적절한 데이터가 요청과 응답에 사용되도록 도움
- 이 기능을 통해 개발자는 별도의 검증 로직을 작성하지 않고도 안정적인 API를 구축할 수 있음

기본 타입 힌트

- FastAPI 에서 경로 매개변수나 쿼리 매개변수에 타입 힌트를 추가하면, 해당 타입에 맞지 않는 요청은 자동 거부
- 또한, 매개변수에 기본값을 설정하여 선택적으로 만들 수 있음

```
# main.py 파일
# 기본 타입 힌트 사용
from fastapi import FastAPI

app = FastAPI()

# 경로 매개변수 예시
@app.get("/items/{item_id}")
def read_item(item_id: int):
    return {"item_id": item_id}

# 쿼리 매개변수 예시
@app.get("/getdata/")
def read_items(data: str = "HelloFastApi"):
    return {"data": data}

# 실행: uvicorn main:app --reload
```

- 브라우저에서 테스트
 - <http://127.0.0.1:8000/items/123>
 - <http://127.0.0.1:8000/items/fun>
 - <http://127.0.0.1:8000/getdata/?data=somequery>
 - <http://127.0.0.1:8000/getdata>
 - <http://127.0.0.1:8000/getdata/?data=1.1>

고급 타입 힌트

- FastAPI 는 typing 모듈에서 제공하는 List, Dict 같은 고급 타입 힌트를 사용하여 요청 데이터를 쉽게 다룰 수 있음

```
# main.py 파일
from fastapi import FastAPI, Query
from typing import List, Dict

app = FastAPI()

# List 데이터 타입을 쿼리 매개변수로 받는 라우트 예제
@app.get("/items/")
async def read_items(q: List[int] = Query([])): # 빈 리스트를 기본값으로 설정
    return {"q": q}

# Dict 데이터 타입을 요청 바디로 받는 라우트 예제
@app.post("/create-item/")
async def create_item(item: Dict[str, int]):
    return item

# 실행: uvicorn main:app --reload
```

- 리스트 자료형 쿼리 매개변수: `List[int] = Query([])`
- 딕셔너리 자료형 요청 본문: `item: Dict[str, int]`

고급 타입 힌트

- 웹 브라우저를 통한 HTTP 클라이언트 테스트

- 목적: FastAPI에서 타입 힌트를 사용하여 데이터 유효성을 검증하는 방법 시연
- List 타입 쿼리 매개변수 테스트

- `http://127.0.0.1:8000/items/?q=1&q=2&q=3`

- 타입 불일치 시 에러 테스트

- `http://127.0.0.1:8000/items/?q=하나&q=둘&q=셋`

- HTTP POST 요청 테스트

- `curl -X POST "http://127.0.0.1:8000/create-item/" -H "accept:application/json" -H "Content-Type: application/json" -d "{\"name\":\"1\"}"`

타입 힌트로 사용 가능한 데이터 타입

- FastAPI에서 사용할 수 있는 파이썬 타입 힌트
- 기본 데이터 타입
 - `int` : 정수
 - `float` : 부동 소수점 숫자
 - `str` : 문자열
 - `bool` : 불리언(`True` 또는 `False`)
- 컬렉션 타입
 - `List` : 변경 가능한 순서가 있는 컬렉션. 즉, `List[int]`는 정수의 리스트를 나타냄
 - `Tuple` : 변경 불가능한 순서가 있는 컬렉션. 즉, `Tuple[str, int]`는 문자열과 정수의 튜플을 나타냄
 - `Dict` : 키와 값의 쌍을 갖는 컬렉션. 즉, `Dict[str, float]`는 문자열 `key`와 부동소수점 숫자 `value`의 딕셔너리를 나타냄
 - `Set` : 중복 없는 항목의 컬렉션. 즉, `Set[bool]`은 불리언 값의 세트를 나타냄

타입 힌트로 사용 가능한 데이터 타입

● 특수 타입

- None : 아무런 값을 갖지 않음을 나타냄
- Any : 모든 타입을 허용. 타입 검사를 무시하고자 할 때 사용

● typing 모듈의 고급 타입

- Optional : 값이 있거나 None일 수 있는 타입. 즉, Optional[str]은 문자열이거나 None일 수 있음
- Union : 여러 타입 중 하나일 수 있는 값. 즉, Union[int, str]은 정수 또는 문자열이 될 수 있음
- Callable : 호출 가능한 객체(함수 등)를 나타냄. 즉, Callable[[int, int], int]는 두 정수 매개변수를 받고 정수를 반환하는 함수
- Iterable : 반복 가능한 객체를 나타냄. 즉, Iterable[str]은 문자열을 항목으로 갖는 반복 가능 객체
- Sequence : 시퀀스 타입을 나타냄. 즉, Sequence[float]는 부동소수점 숫자의 시퀀스

● 사용자 정의 타입

- 클래스나 다른 타입 힌트를 사용하여 사용자 정의 타입을 생성할 수 있음.

즉, 클래스 Person을 정의하고 def get_person() -> Person: 과 같이 사용할 수 있음

HTTP 메소드

: 클라이언트가 서버에게 어떤 동작을 해달라고 요청하는 방식을 정의

● GET

- 용도 : 서버로부터 정보를 요청할 때 사용. 데이터를 가져오는 read-only 작업에 적합
- 예) 사용자의 프로필 데이터나 게시글 목록을 가져올 때 GET 요청을 사용

● POST

- 용도 : 서버에 데이터를 전송하여 새로운 리소스를 생성할 때 적합. 데이터를 서버의 특정 경로에 제출하며, 해당 데이터는 주로 요청 바디에 포함됨
- 예) 새 사용자를 등록하거나 게시글을 작성할 때 사용

● PUT

- 용도 : 지정된 리소스의 전체 업데이트를 수행. 기존 리소스의 완전한 교체를 의미
- 예) 사용자의 전체 프로필을 업데이트하는 경우

● DELETE

- 용도 : 지정된 리소스를 삭제할 때 사용. 서버에 리소스의 제거를 지시하며, 성공적으로 처리된 경우 리소스에 더 이상 접근할 수 없음
- 예) 사용자가 자신의 계정을 삭제하거나 작성한 게시글을 제거하고 싶을 때 사용

주요 HTTP 응답 코드

2xx (성공)

- **200 OK:** 요청이 성공적으로 처리되었습니다. 가장 일반적인 성공 응답 코드입니다.
- **201 Created:** 요청이 성공적으로 이루어져 새로운 리소스가 생성되었습니다. 예를 들어, POST 요청을 통해 새로운 엔트리가 데이터베이스에 추가되었을 때 사용됩니다.
- **204 No Content:** 요청은 성공적이지만, 클라이언트에게 보내줄 콘텐츠는 없습니다. 예를 들어, DELETE 요청이 성공적으로 처리되었을 때 사용될 수 있습니다.

3xx (리다이렉션)

- **301 Moved Permanently:** 요청한 리소스가 영구적으로 새 위치로 이동했습니다. 이 코드는 리소스의 URL이 변경되었을 때 사용됩니다.
- **302 Found:** 요청한 리소스가 일시적으로 다른 위치에 있음을 나타냅니다. 리다이렉션을 위해 자주 사용됩니다.

주요 HTTP 응답 코드

4xx (클라이언트 오류)

- **400 Bad Request:** 서버가 요청을 이해할 수 없음. 잘못된 요청 구조나 파라미터 때문에 발생할 수 있습니다.
- **401 Unauthorized:** 요청이 인증을 필요로 함. 보통 로그인 하지 않은 사용자가 보호된 리소스에 접근하려 할 때 반환됩니다.
- **403 Forbidden:** 서버가 요청을 이해했으나, 권한이 없어 요청을 거부합니다.
- **404 Not Found:** 서버가 요청한 리소스를 찾을 수 없습니다. URL 오류나 존재하지 않는 페이지에 대한 요청에서 흔히 발생합니다.
- **405 Method Not Allowed:** 요청된 리소스에서는 지원하지 않는 HTTP 메소드를 사용했습니다.
- **422 Unprocessable Entity:** 요청은 이해했으나, 요청된 작업을 수행할 수 없음. 주로 요청 형식이 올바르지만, 요청 내용이 유효하지 않을 때 사용됩니다(예: 데이터 유효성 검증 실패).

주요 HTTP 응답 코드

5xx (서버 오류)

- **500 Internal Server Error:** 서버 내부 오류로 인해 요청을 처리할 수 없습니다. 가장 일반적인 서버 오류 응답입니다.
- **503 Service Unavailable:** 서버가 일시적으로 요청을 처리할 수 없습니다. 일반적으로 서버 과부하나 유지 관리로 인해 발생합니다.

Pydantic 모델

- 데이터 유효성 검사와 데이터 직렬화, 역직렬화를 위해 사용되는 클래스
- Pydantic 라이브러리를 활용하여 정의되며, FastAPI의 경로 작업 함수에서 요청 및 응답 데이터의 타입을 명시하는데 주로 사용
- 이 모델들은 타입 힌트를 기반으로 하여 데이터의 형태와 유효성을 선언적으로 정의할 수 있게 해주며, 자동 문서 생성, 요청 검증, 응답 모델링 등에 활용됨
- Pydantic 모델을 사용함으로써 개발자는 데이터에 대한 추가적인 검증 로직을 적게 작성하면서도, 강력한 타입 안전성과 함께 더 깔끔하고 유지보수가 용이한 코드를 구성할 수 있음

Pydantic 모델

● 데이터 검증(Data Validation)

- 정의 : 사용자나 다른 시스템이 보내는 데이터의 형식과 값을 확인하는 과정
- 목적 : 잘못된 데이터 처리 방지, 버그 및 문제 예방
- 예시 : 금액 입력 시 문자열 대신 숫자 사용

● 데이터 직렬화(Data Serialization)

- 정의 : 복잡한 데이터 구조를 바이트나 문자열로 변환하는 과정
- 목적 : 서로 다른 시스템 간의 데이터 교환 용이성 제공
- 역직렬화 : 문자열/바이트를 원래 데이터 구조로 변환

● 중요성

- FastAPI에서는 Pydantic 모델 사용을 권장
- 이유
 - 데이터 검증과 직렬화를 통해 API의 정확성 및 안정성 향상
 - HTTP POST 메소드 같은 데이터 요청에 효과적

Pydantic 기본 문법

● 변수 타입

- 필드 타입 명시 - name:str (변수이름:타입)
- 선택적 필드 : Optional 타입 사용
- 키워드 예시
 - int, float, bool, str, datetime.datetime(날짜와 시간)
 - Optional[str] : typing 모듈의 일부로, 필드가 선택적임을 나타냄. 문자열 또는 None 허용

● Field 제약 조건 : 필드에 추가적인 정보나 제약 조건을 지정할 때 사용하는 함수

- default : 필드의 기본값 설정
- alias : 데이터 전송시, JSON 필드의 이름을 Python 변수와 다르게 설정
- title : 스키마에서 볼 수 있는 필드명으로 주로 문서화에 사용
- description : 필드에 대한 설명
- min_length, max_length : 문자열 길이의 최소값과 최대값을 지정
- gt(greater than), lt(less than) : 숫자 필드의 값 제한

중첩된 모델

- 중첩된 모델

- 하나의 모델이 다른 모델을 포함하는 구조를 의미
- 복잡한 데이터 형태를 모델링할 때 유용

- 예시

- Image 모델이 Item 모델 안에 포함되어 있음
- Item 모델은 Image 타입의 image 필드를 가짐

```
class Image(BaseModel):  
    | url: str  
    | name: str  
  
class Item(BaseModel):  
    | name: str  
    | description: str  
    | image: Image
```

- 중첩된 모델의 장점

- 재사용성 : Image 모델을 여러 모델에서 재사용할 수 있음
- 가독성 : 복잡한 데이터 구조를 더 읽기 쉽게 만들
- 유지보수 : Image 모델을 수정하면 사용된 모든 곳에서 자동으로 반영됨

List와 Union

- List와 Union은 FastAPI와 Pydantic에서 복잡한 데이터 구조와 다형성을 효과적으로 모델링하는 데 사용되는 타입 힌트

● List

- 용도 : 동일 타입의 여러 값들의 배열이나 리스트를 정의
- 문법 : `List[<type>]`
- 예시 : `List[<int>]`는 정수 리스트, `List[<str>]`는 문자열 리스트

● Union

- 용도 : 여러 타입 중 하나를 허용하는 변수를 정의
- 문법 : `Union[<type1>, <type2>, ...]`
- 예시 : `Union[int, str]`는 정수 또는 문자열 가능

FastAPI 응답 모델

- 응답 모델의 개념 및 중요성

- 정의: FastAPI에서 클라이언트에 반환되는 데이터의 구조를 정의하는 기능
- 목적: 데이터의 유효성 보장 및 API 사용자에게 명확한 정보 제공
- OpenAPI 스키마 생성: 자동 문서화에 기여, API 사용자에게 유용한 정보 제공

- 응답 모델의 사용 권장

- 구조 명확화: API가 반환하는 데이터 구조 정의
- 자동 문서 생성: 사용자에게 API 문서 자동 제공
- 데이터 유효성 검증: 자동으로 반환 데이터의 유효성 검사 수행

주요 응답 모델의 종류 및 특징

● 기본 응답 모델

- 정의: Pydantic의 BaseModel을 상속하여 정의하는 가장 일반적인 형태의 응답 모델
- 특징: API 응답으로 사용할 데이터 모델을 명확하게 정의하고, FastAPI 경로 연산에서 response_model로 지정하여 사용

● Union 응답 모델

- 정의: 파이썬의 typing.Union을 사용하여 여러 다른 모델 중 하나를 반환할 수 있는 응답 모델
- 특징: API가 다양한 가능성 중 하나를 선택하여 반환할 때 사용, 유연하고 다양한 응답 처리 가능

● List 응답 모델

- 정의: 리스트 형태의 데이터를 반환할 때 사용되는 FastAPI 응답 모델
- 특징: List 타입 힌트를 사용하여 반환 데이터가 특정 모델을 준수하는 리스트인지 검증, 일관된 데이터 구조 제공

FastAPI 응답 클래스

- FastAPI 응답 클래스의 정의 및 사용 목적
 - 정의: HTTP 응답의 종류를 정의하는 클래스
 - 목적: 반환되는 데이터 형식 제어 및 HTTP 응답의 세부적인 동작 조정
- 주요 응답 클래스 리스트
 - JSONResponse
 - 용도: JSON 형식의 데이터 반환
 - 적용: Python 딕셔너리나 Pydantic 모델을 JSON 문자열로 변환하여 반환
 - HTMLResponse
 - 용도: HTML 형식의 데이터 반환
 - 적용: 웹 페이지의 일부 또는 전체 HTML 마크업 반환에 사용
 - PlainTextResponse
 - 용도: 단순 텍스트 형식의 응답 반환
 - 적용: 로깅 메시지, 간단한 안내 문구 등의 텍스트 반환에 적합
 - RedirectResponse
 - 용도: 클라이언트를 다른 URL로 리디렉션
 - 적용: 사용자를 다른 페이지로 유도하거나 경로를 변경할 때 사용

요청(Request)

- 프로토콜: 컴퓨터나 원격 장치 간 통신을 위한 규칙의 집합. 예) TCP/IP, HTTP, FTP
- HTTP 프로토콜 : 웹에서 데이터 교환을 위한 주요 프로토콜. 클라이언트와 서버 간 HTML 문서나 이미지와 같은 리소스 요청 및 전송에 사용
- HTTP Request: 클라이언트가 서버로부터 정보를 얻거나 서버에 정보를 전달하기 위해 보내는 HTTP 메시지
- HTTP Request의 구성
 - Method: 작업 유형을 정의(GET, POST, PUT 등)
 - URL: 요청이 지시되는 리소스 위치
 - Headers: 요청에 대한 메타데이터(인증, 캐싱, 클라이언트 유형 정보 등)
 - Body: 일부 HTTP 메소드에서 사용되며, 서버에 전송할 데이터 포함
- FastAPI에서의 HTTP Request 정의
 - 주로 Query와 Body() 클래스를 활용하여 세밀한 정의 가능함

요청(Request)

- 쿼리 매개변수(Query Parameter)
 - URL에서 ? 기호 뒤의 키-값 쌍
 - 예시: <http://127.0.0.1:8000/items?id=1> 에서 id=1이 쿼리 매개변수
- FastAPI에서 Query Parameter 다루기
 - 방법: Query 클래스를 사용하여 선언 및 검증 규칙 추가
 - 기능: 함수의 매개변수로 사용하여 Query Parameter임을 명시

Query 클래스의 주요 옵션

- Query 클래스의 역할 및 기능
 - 역할: FastAPI에서 쿼리 매개변수에 대한 메타데이터와 제약조건 지정
 - 기능: 값 전달 이상의 다양한 옵션 제공으로 쿼리 매개변수의 사용성과 안정성 향상
- 주요 옵션 및 사용법
 - 기본 구조: Query(default, 옵션리스트)
 - 주요 옵션
 - default: 파라미터의 기본값. None일 경우 파라미터는 선택적
 - min_length / max_length: 문자열 파라미터의 최소/최대 길이
 - alias: 파라미터의 별칭. URL에서 사용하는 이름과 함수 내에서 사용하는 이름을 다르게 설정
 - deprecated: 파라미터의 사용 중단 여부. API 문서에 경고로 표시
 - description: 파라미터에 대한 설명. API 문서에 표시됨
 - ge / le : 파라미터 값의 최소/최대 범위 지정
 - regex: 파라미터 값의 정규 표현식 패턴 지정
 - title : 파라미터의 설명 제목. 주로 API 문서에서 사용
 - example: 파라미터의 예시 값 제공. 문서에서 예상 입력값을 보여줌

Request Body

- HTTP 프로토콜과 요청

- HTTP(HyperText Transfer Protocol)는 웹에서 데이터를 교환하기 위한 프로토콜
- HTTP request는 클라이언트가 서버에 특정 작업을 요청하는 메시지
- HTTP 요청(request)
 - Method: 서버에 요청하는 작업의 유형을 정의((예) GET, POST, PUT)
 - URL: 요청이 지시되는 리소스의 위치를 나타냄
 - Headers: 요청에 대한 메타데이터를 포함하며 인증, 캐싱, 클라이언트 유형 등의 정보를 담음
 - Body: 일부 HTTP 메소드(POST, PUT)에서 사용되며, 전송할 데이터를 담음

- FastAPI에서의 Request Body 처리

- 요청 바디(request body)는 클라이언트가 서버로 전송하는 데이터의 본문
- 주로 POST, PUT, PATCH 메소드를 사용할 때 볼 수 있으며, 서버가 수행해야 할 상세한 작업이나 서버에 제출할 데이터를 포함
- Body() 함수를 사용하여 복잡한 구조의 데이터를 처리하고 관리
- 예) 사용자가 웹 폼을 통해 데이터를 전송할 때, 그 데이터는 요청 바디에 포함되어 서버로 전송됨

Request Body

- Request Body 처리 방식
 - 적용 메소드: POST, PUT 메소드에서 주로 사용됨
 - GET 요청: 일반적으로 Body를 포함하지 않으며, Query Parameters 또는 URL 경로를 통해 데이터 전송
- FastAPI의 Body() 함수의 옵션
 - default: 필드의 기본값 설정. 선택적 필드로 만들기 위해 None 사용
 - example: 문서에서 보여줄 예시 값 설정
 - media_type: Request Body의 미디어 타입 명시(예: application/json)
 - alias: 필드의 별칭 설정(실제 사용되지 않음)
 - title: 문서에서 보여줄 Body의 제목
 - description: Body에 대한 상세 설명
 - deprecated: 필드의 사용 중단 여부 표시
- FastAPI의 Body() 함수와 옵션들은 API 설계를 더 정밀하고 사용자 친화적으로 만들어 주어, 개발자는 이를 통해 복잡한 데이터 구조와 요구사항을 쉽게 처리하고, 효과적인 API 문서화를 제공할 수 있음

예외 처리(Exception Handling)

- FastAPI의 예외 처리 개념

- 목적: 예상치 못한 오류나 예외 상황에 대응하는 프로세스
- FastAPI의 특징: Python의 기본 예외 처리 메커니즘을 활용하며, 웹 어플리케이션에 특화된 예외 처리 기능 제공

- 기본 예외 처리 방법

- 사용법: Python의 try / except 문법 사용
- 과정: try 블록에서 예외 발생 시 except 블록으로 제어 이동, 오류 처리 실행

- FastAPI의 HTTPException 클래스

- 목적: API에서 발생하는 예외를 명확하게 알리기 위한 기능 제공
- 기능: 다양한 에러 상황에 대해 HTTP 상태 코드와 에러 메시지 정의 및 반환

- HTTPException의 주요 옵션

- status_code: HTTP 상태 코드 설정, 요청의 성공, 실패 등을 나타냄. 예) status_code=404
- detail: 에러에 대한 상세 메시지 제공. 예) detail="item not found"